

14.2 ReAct and Tool Use

Thought, action, observation. The model calls a tool instead of guessing the lookup.

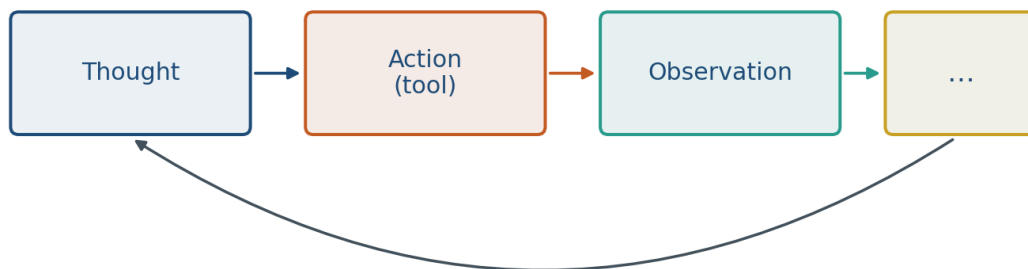
These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

ReAct (Yao et al.) interleaves **Thought**, **Action**, and **Observation**. The model is not only writing a chain of thought. It can call a tool, read the result, and then continue.

1. The loop

A thought is scratch work. An action is a structured call: `search["library hours"]`, `calc[8+15]`, `lookup[d3]`. The environment returns an observation (a string). You append it to the context and sample again until a final answer.

ReAct: think, call a tool, read the result, repeat



RAG (note **14.1**) is one tool: retrieve. A calculator is another. A code runner is another. The control pattern is the same. Lab 14's calculator is dummy arithmetic so you can implement the loop without an API. The observation must appear in the context; a thought that ignores it is the same bug as unfaithful CoT.

2. Why tools change the error modes

CoT can invent a product of two numbers. A calc tool can return the product if the model **writes the right call**. New failures: malformed actions, calling the wrong tool, ignoring the observation, or looping. That is still unfaithfulness: the thought says "I added" when the observation was never used.

Parse actions strictly (regex or JSON). Cap the number of steps. Log every observation in the report appendix for one worked example. If the parse fails, return an error observation and let the model retry once; do not silently drop the call.

3. ReAct versus ToT

ToT searches among thoughts. ReAct calls the world. You can combine them (search over tool-using traces), but name the knobs separately. For the project, one tool plus a checker is enough; a zoo of APIs is not.

4. Practice

1. Write one Thought–Action–Observation triple that answers “what is 17×24 ?” with a calculator, without doing the multiply in the thought.
2. The observation is `23:00` and the model answers `midnight`. What broke?
3. Why is an unconstrained `python_eval` tool riskier than a dummy `calc` that only does `+` and `*` on numbers?